



**Faculty of Mechanical
Engineering and Robotics**

**Department of Robotics and
Mechatronics**



Signal processing and identification in control of mechatronic devices

Topic: Image preprocessing – filtering and edge detection

Goal: Students are introduced to basic linear and non-linear image filtering methods. An application of image filtering for noise reduction and edge detection is discussed.

Problems:

1. Basics of 2D linear filtering,
2. Low-pass filtering and noise reduction,
3. Non-linear filters application for noise reduction,
4. High-pass filtering and edge detection applications.

Prowadzący: dr inż. Krzysztof Holak

1. Image Filtering basics in MATLAB.

Filtering is one of the major image processing context operations. Linear filters are implemented as a convolution of an image with masks of different numerical coefficients, depending on an application. Filters can be divided into low-pass and high-pass ones. Low-pass filters remove high frequency components of the image signals, and high-pass filters removes frequency content which is less than some threshold frequency.

In MATLAB, there is implemented function **conv2()** which computes convolution of two 2-D signals, such as images. The arguments of the function are image and specified filter mask. The function returns filtered image as an output. The result is in double, therefore you need to convert it to uint8 before showing on the screen. An example:

```
im = imread('Obrazek.jpg');
mask = (1/9)*[1 1 1; 1 1 1; 1 1 1];
im_filt = conv2(double(im), double(mask));
imshow(uint8(im_filt));
```

Problem 1. Test image filtering using different masks of low-pass and high-pass filters taken from the literature. How chosen filters change an image? What do the high and low spatial frequencies represent in the case of images?

2. Low-pass filtering and nonlinear filters.

MATLAB has functions dedicated for image filtering implemented. The function does not require specifying the exact coefficients of a mask. In order to create a mask of a given filter, you need to provide type of a filter, size of the mask and additional parameters. An example of the function usage:

```
im = imread('Obrazek.jpg');
h = fspecial('average', [3 3]);
im_filt = imfilter(im,h,'replicate');
imshow(uint8(im_filt));
```

The program works exactly like the one implemented in the previous point.

Low-pass filtering is implemented by the following filters:

1. Square averaging filter of a size given by the variable 'hsize'
`h = fspecial('average', hsize);`

2. Circular averaging filter of a size given by the variable 'radius'.
`h = fspecial('disk', radius);`
3. Gaussian filter which takes the size of the mask and standard deviation of a Gaussian curve as inputs.
`h = fspecial('gaussian', hsize, sigma);`

Non-linear filtering:

The most common nonlinear filter in the image processing is median filter. It computes a median value of intensities of pixels under the mask passing through an image. In MATLAB, median filtering is computed by function **medfilt2()**.

An example of an application of median filter for salt and pepper noise removal.

```
im = imread('Obrazek.tif');  
im_noise = imnoise(im, 'salt & pepper', 0.02);  
im_filtered = medfilt2(im_noise);  
imshow(im_noise), figure, imshow(im_filtered)
```

In the example, **imnoise()** function was applied in order to add noise of a given statistical characteristics.

Problems 2.

- Test different types of low-pass filters. What is the influence of mask size on the result of filtering? What is the difference between averaging and Gaussian filter?
- There is an extra parameter in function **imfilter()**. In the example, the value of the parameter was 'replicate'. What is the meaning of the parameter?
- Improve the quality of images given by the lab assistant. Chose appropriate filters for different types of noise
- Using function **imnoise()**, check which filters are applicable for different types of noise characteristics

3. High-pass filters. Edge detection.

There are many different methods for an edge detection. In case of lab session, we will analyze the ones based on high-pass filtering. The filters compute first and second derivatives of an image numerically. The edges are found in the pixels for which the response of the filter was the highest, in case of first derivative, or in the zero crossing in the case of second derivative filters.

In order to detect edges, function known from chapter 2 can be used:

1. Filters computing first and second derivatives in x and y directions. There are two types of filters provided: Prewitt and Sobel operators.

```
h = fspecial('prewitt');  
h = fspecial('sobel');
```

2. Filters computing second derivatives. In MATLAB, there are two types of filters – Laplacian and Laplacian of a Gaussian (LoG). These filters compute the sum of second derivatives of an image in x and y directions.

```
h = fspecial('laplacian',alpha);  
h = fspecial('log',hsize,sigma);
```

3. Special filter applicable for image sharpening. h

```
= fspecial('unsharp',alpha);
```

As an alternative to the methods mentioned above, function **edge()** can be applied. To apply a Sobel filter, use the following code:

```
im_edge = edge(im,'sobel',thresh,direction);
```

The parameter 'thresh' determines a response of a filter above which a pixel is considered to belong to an edge. The parameter 'direction' gives the direction of computed derivative ('horizontal' or 'vertical'). New features are Roberts filter and Canny edge detection method, one of the most important edge detection algorithm in computer vision field.

Problems 3.

1. Look at the masks returned by functions fspecial for Prewitt and Sobel operators. Which types of edges they detect? Check this experimentally. How can you obtain a filter which returns edges that are perpendicular to the ones computed by the filter you analyzed previously?
2. What is the difference in images processed by first and second order derivative filters?
3. What is the difference between LoG filter and Laplacian operator. What is the meaning of sigma parameter?
4. Test and describe the impact of unsharp masking on the image.
5. Write a code and test filters implemented in edge() function. Detect edges directionally and check what is the performance of Roberts filter and Canny algorithm. What is the main advantage of Canny method over all previous edge detection filters.